

International Journal of Web Engineering & Business Information Systems
June 2026

ProTech: A Containerised Django Marketplace for Professional Services with Escrow Payments, RBAC, and Two-Sided Monetisation

An End-of-Year Master's Project in IT Innovation in Business

Manye Patrice

IT Innovation in Business — Master's Degree

Web Technology in Business · Web Application Development

Email: logandescapo@gmail.com | Repository: github.com/logandescapo-eng/protech1

Abstract

This paper presents ProTech, a full-stack professional services marketplace developed as an end-of-year master's project within the IT Innovation in Business programme. The platform connects clients with skilled service workers through a unified web application supporting discovery, booking, messaging, post-job reviews, and escrow-protected payments. Unlike an earlier PHP/MySQL prototype, the current system uses Django 5.2.15, PostgreSQL 15, Redis 7, Gunicorn, and Nginx via Docker Compose. Security is enforced through PBKDF2 hashing, CSRF middleware, ORM parameterisation, template auto-escaping, and a custom `@user_type_required` decorator. The escrow subsystem in `escrow/services.py` manages wallets, held funds, platform commission (default five percent), and auditable transactions using Decimal arithmetic and atomic database transactions. The paper covers architecture, business model, data design, DevOps, testing (seventeen automated tests, GitHub Actions CI), limitations, and future work.

Keywords: Django, marketplace platform, escrow payments, role-based access control, Docker Compose, PostgreSQL, Redis caching, two-sided markets, full-stack web development.

1. Introduction

The digital transformation of professional services has created substantial demand for platforms that connect clients with verified workers while reducing search friction, payment risk, and administrative overhead. Established marketplaces such as TaskRabbit, Thumbtack, and Upwork demonstrate that two-sided platforms can aggregate supply and demand at scale; however, local tradespeople often lack affordable channels to build reputation and receive secure online payments.

ProTech addresses this gap through an academically rigorous yet commercially informed implementation. Developed by Manye Patrice as the capstone for Web Technology in Business and Web Application Development, the platform serves clients who browse, book, and pay for services, and workers who manage profiles, accept jobs, and receive wallet-based payouts. The project evolved from a PHP/MySQL prototype into a Django-centred system aligned with enterprise web practice and the module rubric.

This article contributes four elements: (1) documentation of migration from PHP to domain-partitioned Django with a Jinja2 compatibility layer; (2) an internal escrow ledger design for marketplace trust; (3) explicit rubric-to-implementation mapping; and (4) a hybrid monetisation model grounded in two-sided market theory.

2. Related Work

Research on electronic commerce adoption emphasises perceived usefulness, ease of use, and trust as primary drivers of platform participation (Pavlou & Fygenson, 2006). Trust mechanisms—verified identity, reputation systems, and payment holding—reduce transaction risk (Edelman et al., 2012). ProTech operationalises these through role-separated dashboards, booking-linked reviews, and escrow holds released only upon job completion.

Platform economics literature formalises pricing on both sides of a market (Rochet & Tirole, 2003; Evans & Schmalensee, 2016). ProTech's design—free client access, worker commissions, optional subscriptions—follows asymmetric pricing common among services marketplaces.

Fielding (2000) and Fowler (2002) inform separation of presentation, domain logic, and persistence. Django's ORM, migrations, admin, and security middleware embody enterprise patterns. Docker Compose reflects reproducible DevOps practice. Security references OWASP Top Ten (2021): RBAC decorators, ORM queries, and PBKDF2 hashing address access control, injection, and cryptographic failures.

3. System Architecture

ProTech adopts a three-tier architecture deployed as four Docker Compose services: PostgreSQL (db), Redis (redis), Django/Gunicorn (backend), and Nginx (nginx). The presentation tier delivers server-rendered HTML via Jinja2 templates. The application tier partitions logic into users, bookings, reviews, and escrow applications. The data tier combines PostgreSQL 15 with Redis 7 for sessions and

query caching. Django middleware enforces CSRF, authentication, and security headers on every request.

Tier	Technology	Responsibility
Presentation	Jinja2 + Nginx	UI rendering, static/media offload
Application	Django 5.2 + Gunicorn	Business logic, auth, admin
Persistence	PostgreSQL 15	Users, bookings, escrow ledger
Cache / sessions	Redis 7	Worker search cache, sessions
Orchestration	Docker Compose	Reproducible deployment

Table 1: Three-tier architecture and deployment components

4. Business Model

ProTech implements a hybrid model: transaction commissions at escrow release and tiered worker subscriptions for visibility. `release_escrow()` applies `ESCROW_PLATFORM_FEE_PERCENT` (default five percent). Tiered rates are specified for micro, standard, and commercial job bands. Subscription plans—Starter (free), Professional (\$29/mo), Premium (\$59/mo)—are documented but billing is not yet coded.

Plan	Price	Visibility	Commission discount
Starter	Free	Basic listing; capped bookings	None
Professional	\$29/mo	Priority above Starter	–1% (4% effective)
Premium	\$59/mo	Featured placement	–2% (3% effective)

Table 2: Worker subscription tiers (specified; billing planned)

5. Authentication and Security

Authentication uses Django sessions with a custom EmailBackend. Passwords are PBKDF2-hashed. Registration creates client Users or User+Worker pairs. Authorisation uses `user_type` and `@user_type_required` to separate client, worker, and admin flows.

Threat (OWASP)	Control	Implementation
----------------	---------	----------------

Broken access control	RBAC decorator	users/decorators.py
Injection	ORM parameterisation	Django ORM
Cryptographic failures	PBKDF2 hashing	Django auth
CSRF	CsrfViewMiddleware	csrf_field() in templates
XSS	Auto-escaping	Jinja2 templates
Misconfiguration	Environment secrets	.env + .gitignore

Table 3: Security controls mapped to OWASP categories

5.1 Contrast with Prior PHP Prototype

The earlier PHP/MySQL prototype used bcrypt, MySQLi prepared statements, and PHP sessions in XAMPP. It lacked admin tooling, migrations, escrow, Docker deployment, automated tests, and caching. The Django rewrite consolidates these capabilities and replaces demo-mode URL bypasses with seed_demo and Docker entrypoint seeding.

6. Escrow Payment Subsystem

ProTech holds client funds in escrow until work is complete. Logic resides in escrow/services.py. Workflow: deposit to UserWallet → create Booking → fund_escrow() creates EscrowHold → release_escrow() on completion (fee deducted) or refund_escrow() on cancellation.

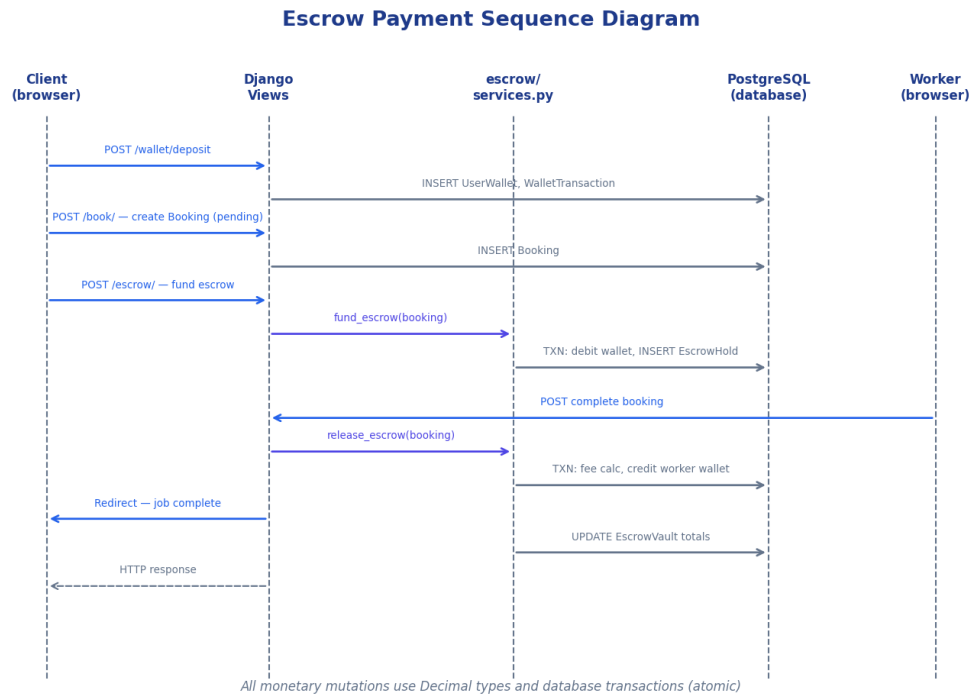


Figure 1: Escrow payment sequence diagram

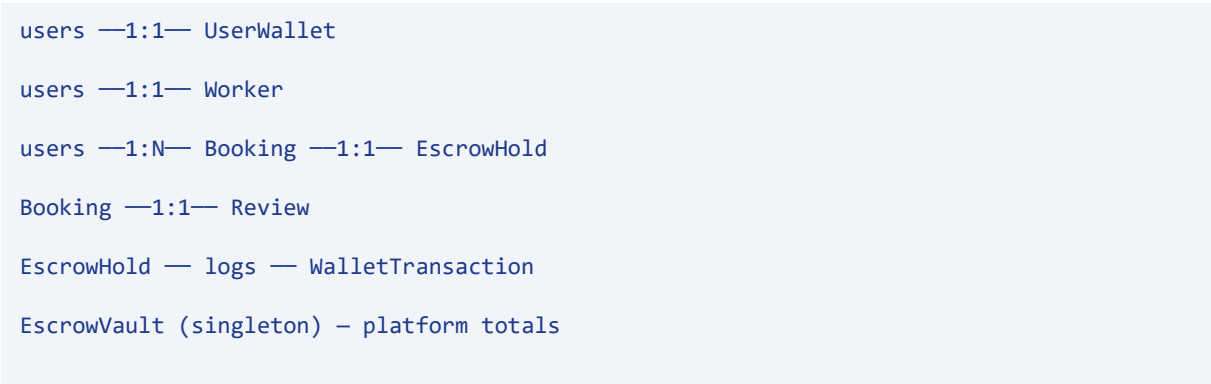


Figure 2: Core entity relationships (textual ER summary)

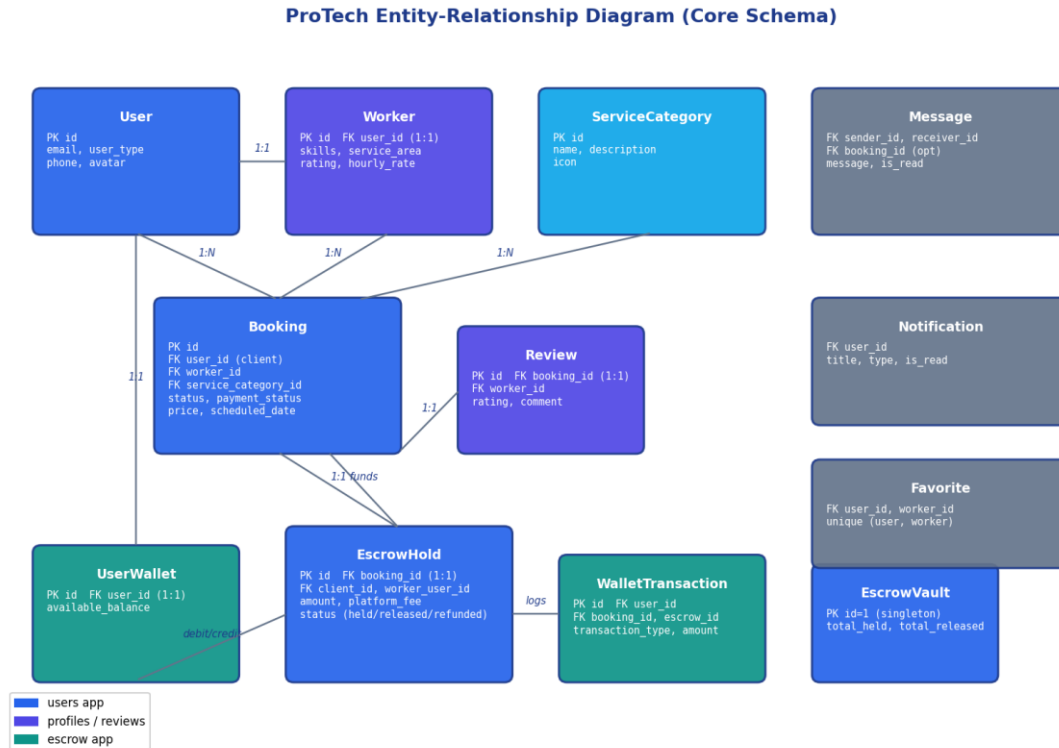


Figure 3: Entity-relationship diagram

7. Data Model and Booking Workflow

The schema centres on User, Worker, ServiceCategory, and Booking. Review attaches one-to-one to completed bookings. Message, Notification, Favorite, and WorkerAvailability support engagement.

Booking status	Actor	Escrow interaction
pending	Worker accepts/rejects	Client may fund escrow
confirmed / in_progress	Worker updates	Funds held if funded
completed	Worker marks done	release_escrow(); fee deducted
cancelled	Either party	refund_escrow() if held

Table 4: Booking states and escrow coupling

8. Implementation and DevOps

Code is organised by domain: users/, bookings/, reviews/, escrow/, protech_project/. cache_utils.py caches worker search (300s) and categories (600s). Logging writes to stdout and logs/django.log. docker-entrypoint.sh runs migrate, collectstatic, seed_demo, and Gunicorn.

Rubric requirement	Implementation	Evidence
Django + templates	Django 5.2 + Jinja2	templates/, jinja2_env.py
Auth & RBAC	@user_type_required	users/decorators.py
Password reset	Django token flow	/password-reset/
Admin / CMS	Django Admin	/admin/
Docker + Redis	Four-service Compose	docker-compose.yml
Tests + CI	17 passing tests	ci.yml
Logging	Per-app loggers	settings.py

Table 5: Academic rubric alignment

9. Testing and Evaluation

Unit tests validate models; integration tests cover login, RBAC, password reset, health endpoint, and escrow wallet debits. GitHub Actions runs flake8, migrations, full test suite, and /health/ check.

Metric	Result	Benchmark
Automated tests	17 passing	Module requirement met
CI pipeline	Lint, migrate, test, health	GitHub Actions
Deployment	docker compose up --build	Single-command demo
Escrow commission	5% default	Live in services.py
Cache	Redis / locmem (CI)	Environment-aware

Table 6: Evaluation metrics and outcomes

10. Limitations

ProTech uses demonstration wallet deposits, not a licensed payment provider. Subscriptions are specified but not billed. Messaging is not real-time. Browser E2E tests are absent. Growth projections are illustrative. Legacy Flask/PHP files remain for reference; Django is canonical.

11. Conclusion and Future Work

This paper presented ProTech as a production-oriented marketplace replacing a PHP prototype with Django, PostgreSQL, Redis, and Docker. The platform integrates discovery, booking, messaging, reviews, escrow, caching, logging, testing, and CI into a system suitable for master's assessment.

Future work prioritises Stripe integration, WorkerSubscription billing, WebSocket notifications, Selenium/Cypress E2E tests, geolocation matching, and two-factor authentication. The modular architecture supports these increments without fundamental restructuring.

References

- [1] Django Software Foundation. (2024). Django Documentation (v5.2). <https://docs.djangoproject.com/>
- [2] Docker Inc. (2024). Docker Compose Specification. <https://docs.docker.com/compose/>
- [3] Edelman, B., Luca, M., & Svirsky, D. (2012). From niches to riches. Harvard Business School Working Paper.
- [4] Evans, D. S., & Schmalensee, R. (2016). Matchmakers: The New Economics of Multisided Platforms. HBR Press.
- [5] Fielding, R. T. (2000). Architectural Styles and the Design of Network-based Software Architectures. UC Irvine.
- [6] Fowler, M. (2002). Patterns of Enterprise Application Architecture. Addison-Wesley.
- [7] OWASP Foundation. (2021). OWASP Top Ten. <https://owasp.org/www-project-top-ten/>
- [8] Pavlou, P. A., & Fygenson, M. (2006). MIS Quarterly, 30(1), 115–143.
- [9] PostgreSQL Global Development Group. (2024). PostgreSQL 15 Documentation.
- [10] Redis Ltd. (2024). Redis Documentation. <https://redis.io/docs/>
- [11] Rochet, J.-C., & Tirole, J. (2003). Platform competition in two-sided markets. JEEA, 1(4), 990–1029.
- [12] Marcotte, E. (2010). Responsive web design. A List Apart, 306.